ILG MVP —Solution Approach

Use LiteLLM standalone (not via LangChain wrapper). Here's why:

LiteLLM standalone gives you a unified OpenAI-compatible interface that calls Claude, GPT-4, Gemini, LLaMA — all with the same API signature. You swap providers by changing one config line. It handles retries, fallbacks, cost tracking, and rate limiting out of the box.

LangChain's LiteLLM wrapper adds LangChain's abstraction layer on top, which introduces unnecessary complexity, version dependency hell, and debugging difficulty for a production system. LangChain is excellent for rapid prototyping but in a production pipeline with custom agents, you end up fighting the framework more than building your solution.

The industry best practice recommendation:

Use **LiteLLM + LangGraph** — not full LangChain. LangGraph is the agent orchestration layer from the LangChain team but it is a separate, much leaner library specifically built for building stateful multi-step agents. It gives you the agent workflow control you need without LangChain's heavy abstractions.

So the combination is: **LiteLLM for LLM provider abstraction + LangGraph for agent orchestration + FastAPI for API layer.**

This is what companies like Replit, Notion, and enterprise teams are using in 2025 for production AI pipelines.

Complete MVP Solution Architecture —

Foundational Principle

The architecture separates three concerns cleanly: **Document Understanding** (Claude's job), **Business Logic** (deterministic Python code's job), and **Reasoning & Orchestration** (LangGraph agent's job). Mixing these is the most common mistake in AI system design.

Layer 1 — Document Ingestion Service

When a document arrives (Selection Sheet PDF, Takeoff Sheet Excel/PDF), it goes to S3 immediately. A Celery worker picks up the job asynchronously via Redis queue. This means your API returns instantly and processing happens in the background. The document never gets

processed synchronously in the HTTP request cycle — that is industry standard for document-heavy systems.

The Celery worker calls the **Document Classification Service** first. This is a lightweight step that identifies: is this a selection sheet or a takeoff sheet? Which builder format is it? This classification uses a small prompt to Claude via LiteLLM and stores the classification result in PostgreSQL.

Why classify first: Your extraction prompts for a Ryan Homes selection sheet are completely different from a DR Horton sheet. The classifier routes to the correct extraction prompt template stored in your database.

Layer 2 — Claude Extraction Pipeline (via LiteLLM)

Selection Sheet Extraction:

You pass the full document to Claude Sonnet 4 via LiteLLM with structured output enforcement. Claude Sonnet 4's extended thinking capability is valuable here for complex multi-page documents where options span pages and change orders override earlier selections

The prompt instructs Claude to output a strict JSON schema: lot information, customer info, selections array where each item has option_code, description, category (B/E/S/R), quantity, color, location_number, and change_order_status (was this added/modified by a change order?).

The change order detection is critical — in the sample document, ICD (Dryer Gas Hook-Up) was added and ICC (Dryer Electric) was deleted in Change Order #1. Claude needs to reconcile the final state, not just list all options blindly.

After extraction, store raw Claude response AND parsed JSON in PostgreSQL. Always store the raw response — when something goes wrong in production, you need the original Claude output to debug without re-calling the API.

Takeoff Sheet Extraction:

Takeoff sheets are tabular Excel-like structures. Claude Sonnet 4.5 is actually better here because the data is more structured and 4.5 is faster and cheaper for structured table extraction.

You send the document and ask Claude to extract each row as a structured record: room_name, std_material, base_opt_elev flag, subfloor, material_width, cut_length, sq_yards, pad_sq_yards,

wood_tile_sqft, shoe_base_lf, cabinet_sides_lf, toe_kick_lf, nosing_lf, threshold_lf, t_molding_lf, option_code, notes.

The option_code field is the foreign key that links back to the selection sheet. This linkage is deterministic — no AI needed for the join.

Layer 3 — Mapping Engine (Pure Python, No AI)

Selection-to-Takeoff Mapping is pure database logic: customer selected option "MAS" (Study) → query takeoff_data WHERE option_code = 'MAS' AND builder_id = 'RYAN' AND plan = 'PISA_TORRE' → get quantities. This is a SQL JOIN. No LLM needed.

Option Reconciliation Logic: When a customer selects HH6 (Hard Surface Upgrade - Great Room), that means the base house carpet for the Great Room is replaced by LVP. The takeoff sheet has separate rows for base and optional configurations. Your mapping engine needs to understand this substitution logic. This is implemented as Python business rules loaded from a configuration table in PostgreSQL — not hardcoded, not AI, but configurable rules that ILG analysts can update via an admin UI.

Quantity Aggregation: Multiple rooms may use the same material type. The engine aggregates: total LVP square footage across Foyer + Kitchen + Great Room + Owner's Bedroom + Dinette + Hall = 1,079 FT2 (matching the Sundry Labor document). This is pure arithmetic.

Sundry Calculation: Every material type has associated sundry items. LVP installation requires masking tape, corrugated paper roll, shoe molding. The ratios are known: 50 linear feet of shoe molding per order, 1.25 EA of corrugated paper per LVP job, etc. These ratios come from ILG's historical data and are stored in a sundry_rules table in PostgreSQL. This is completely deterministic.

Layer 4 — SAP Material Matching (pgvector + Confidence Scoring)

Embedding Strategy:

When you seed the 17,000-row SAP Excel into PostgreSQL, you generate embeddings using a dedicated embedding model — not Claude. Use **text-embedding-3-small** from OpenAI via LiteLLM or **nomic-embed-text** (open source, runs locally). Embedding models are much cheaper than Claude for this purpose and produce better embeddings for semantic similarity than using Claude's responses.

Each SAP material description gets embedded and stored in the pgvector column. The material category and trade type are stored as filter columns.

Matching Logic:

When you have "LVP Visionary Taupe AHF" from the takeoff, you generate its embedding and run a pgvector cosine similarity search filtered by trade_type = 'LVP'. You get top 5 matches with similarity scores.

The confidence tiers are: above 0.92 means auto-map and log it; between 0.75 and 0.92 means present to human analyst as suggested match requiring one-click confirmation; below 0.75 means present top 5 candidates and require manual selection. Every human decision is stored and used to build a confirmed_mappings cache table — next time "LVP Visionary Taupe" appears, you check the cache first before running vector search.

Over time the cache grows and 80% of lookups become instant cache hits. This organically reduces both latency and cost.

Layer 5 — LangGraph Agent for Purchase Order Generation

Why LangGraph specifically:

LangGraph lets you define your agent as a stateful graph with nodes and edges. Each node is a processing step. Edges can be conditional — if validation fails, route back to correction node; if human review needed, pause and wait for input. This is exactly the human-in-the-loop pattern ILG needs.

The Agent Graph Structure:

The graph has these nodes: Initialize (load all mapped data for a lot), ValidateSelections (check for contradictions between selection sheet and takeoff), GenerateOrderLines (create SAP order lines for installation materials), GenerateSundryLines (add sundry items per business rules), GenerateLaborLines (add labor SAP codes per material type), ValidateQuantities (check against historical ranges — flags if LVP quantity for a 3-bedroom is 5,000 FT2 which is implausible), HumanReview (pause and present to analyst), and FinalizeOrder (generate SAP-upload Excel).

The LiteLLM Call within the Agent:

Claude is called via LiteLLM only at the ValidateSelections and ValidateQuantities nodes — where reasoning about contradictions and anomalies requires intelligence. The GenerateOrderLines, GenerateSundryLines, and GenerateLaborLines nodes are pure Python

using the rule tables from PostgreSQL. Claude is not called for generating routine order lines — that would be wasteful and non-deterministic.

This is the industry best practice: LLM for reasoning and anomaly detection, deterministic code for calculation and generation.

The Agent System Prompt to Claude:

The system prompt contains ILG's business rules in structured form: material substitution rules (if carpet selected but LVP option also selected, LVP takes precedence for that room), quantity sanity checks (typical 3-bed home LVP range is 800-1400 FT2), sundry attachment rules per material type, and labor code mapping by material category.

Claude returns structured JSON flagging any issues, explaining the contradiction in plain English (for the analyst to understand), and suggesting resolution.

Layer 6 — Human-in-the-Loop Validation UI (Next.js)

Five Review Screens corresponding to the five workflow steps:

The Selection Review screen shows the original document on the left (PDF viewer) and extracted JSON data on the right as an editable form. Analysts can correct any field. The document is highlighted to show which section each extracted field came from.

The Takeoff Review screen shows a table of rooms with computed quantities. Analysts can adjust quantities before SAP mapping.

The SAP Mapping Review screen shows each material with its matched SAP code, similarity score as a visual confidence bar, and the description. Low-confidence matches are highlighted in amber. Analysts click to confirm or select from alternatives.

The Order Preview screen shows the full proposed purchase order in the same format as the Sundry Labor Calculation document you shared. Analysts can edit any line.

The Final Approval screen requires sign-off and generates the SAP-upload Excel file.

Every edit is captured. When an analyst corrects an extracted field, that correction is stored as a corrections record with: original_value, corrected_value, field_name, document_type, builder_id, correction_by, correction_timestamp. This is your future training data and your audit trail.

PostgreSQL Schema (Key Tables)

The core tables are: documents (raw document metadata and S3 path), document_classifications (builder, format, document type), selections (extracted selection data per lot), takeoff_data (extracted takeoff quantities), takeoff_mapped (selection-to-takeoff cross-referenced data), sap_materials (17k rows with pgvector embedding column), confirmed_mappings (cache of human-validated material matches), sap_material_changelog (version history), order_drafts (agent-generated PO in progress), order_lines (individual line items), corrections (all human corrections), sundry_rules (material-to-sundry attachment rules), and labor_rules (material-to-labor code mapping).

Critical Gaps That Must Be Addressed

Gap 1 — Prompt Template Management

You need a prompt_templates table in PostgreSQL with columns: template_id, builder_id, document_type, version, prompt_text, is_active, created_at, created_by, performance_metrics (JSONB).

Why this matters practically: When Claude misextracts something from a Ryan Homes sheet, an analyst needs to update the prompt for that builder without touching code and without a deployment. The prompt management UI in your admin panel is what enables this. Without it, every prompt fix requires a developer and a deployment cycle.

You also need prompt versioning so when you update a prompt, old jobs can be reprocessed with the new version, and you can compare extraction accuracy between versions.

Gap 2 — The Takeoff Sheet Logic

The solution correctly identifies that selection-to-takeoff mapping is a SQL JOIN on option_code. But it misses a critical complexity that actual documents reveal.

Looking at the takeoff sheet: when a customer selects HH6 (Hard Surface Upgrade - Great Room), the carpet row for Great Room (315 sq yards, 41.00 cut length) is REPLACED by the LVP row for Great Room. But the takeoff sheet has both rows. Your mapping engine needs to know which rows are active for this specific customer.

This requires a **Material Substitution Matrix** table: when_option_selected, replaces_material_type, in_room, with_material_type. For example: when HH6 is selected, replace CARPET in GREAT_ROOM with LVP.

This matrix is the core business logic of ILG's domain knowledge. It needs to be populated from ILG's existing analyst knowledge, stored in PostgreSQL, and have an admin UI for maintenance. This is not something Claude can derive — it is explicit business rules that ILG analysts know.

Without this matrix, your mapping engine will either double-count materials (carpet + LVP for the same room) or miss substitutions entirely, which would produce wrong purchase orders.

Gap 3 — Error Handling and Retry Strategy is Not Defined

The solution describes the happy path but production document processing fails constantly. PDFs are corrupted, scanned documents have poor quality, Claude occasionally returns malformed JSON, network calls timeout.

You need a defined retry strategy per failure type. Claude extraction failures should retry up to 3 times with exponential backoff, and if all retries fail, the job moves to a failed_jobs queue with the error captured, and an analyst is notified to manually review. Celery has dead letter queue support natively — this must be configured explicitly.

Partial extraction failures are more subtle. Claude might successfully extract 85% of a selection sheet but miss a section. Your validation layer needs to check completeness: does the extracted JSON have entries for all expected categories (flooring, electrical, plumbing, etc.) for this builder's known schema? Missing categories should flag for human review, not silently proceed.

Gap 4 — The Audit Trail

The corrections table captures analyst edits but the full audit trail for compliance and debugging needs more.

You need an audit_events table that records every state transition for every document job: document uploaded, classification started, classification completed, extraction started, extraction completed (with token count and cost), validation passed or failed, human review triggered, human approved, order generated, order exported. Each event has timestamp, user_id if human action, duration_ms, and metadata JSONB.

This is not optional for an enterprise system like ILG. Their compliance team will ask "who approved this order and when" and "why was this SAP code chosen for this material." You need to answer that from the database without reconstructing from logs.

Gap 5 — Multi-Builder Configuration Architecture is Underspecified

The POC is one builder but the solution is supposed to scale to multiple builders. The current schema has builder_id as a column but doesn't define the full builder configuration model.

Each builder needs a configuration record that contains: builder_id, builder_name, selection_sheet_format (RYAN_V3, DR_HORTON_V2, etc.), takeoff_sheet_format, active_prompt_template_ids per document type, plan_codes (PISA_TORRE, etc.), option_code_prefix patterns, and contact_info.

This configuration drives everything: which prompt template to use, which substitution matrix applies, which sundry rules apply (different builders may have different standard sundry attachments). Without this configuration model, adding a second builder requires code changes. With it, adding a builder is a data operation done in the admin UI.

Gap 6 — The Excel Output Generation is Not Designed

The solution mentions "generate SAP-upload Excel" as the final step but doesn't specify how. This is actually a precise requirement — the Excel must match ILG's SAP upload template exactly, otherwise SAP rejects the import.

Use the openpyxl library in Python. You need a template Excel file stored in S3 that defines the exact column headers, formatting, and sheet structure that SAP expects. Your order generation step fills this template programmatically from the order_lines data. The filled template is saved back to S3 and a download link is provided in the UI.

You also need an Excel validation step before download: check that all required SAP columns are populated, UOM values are valid SAP UOM codes, SAP material codes are valid (exist in your sap_materials table), and quantities are positive numbers. This prevents uploading a malformed file to SAP which can corrupt live procurement data.

Gap 7 — Security and Access Control

At minimum for MVP need: JWT-based authentication on all FastAPI endpoints, role-based access control with at minimum three roles (Analyst who can review and approve, Admin who can manage configurations and prompt templates, Viewer who can only see order status), and row-level security so analysts only see lots they are assigned to.

For document storage: S3 bucket must have private ACL with pre-signed URLs for document access (never public URLs). All documents contain customer PII (customer names, addresses, lot numbers).

Database connections must use SSL. Environment variables for all secrets, never hardcoded. This is table stakes for any enterprise system handling customer data.

Gap 8 — Observability

LiteLLM dashboard for costs and Flower for Celery are mentioned but production observability requires more.

You need structured logging across all services using a consistent format (JSON logs with job_id, lot_id, builder_id, step_name on every log line). This lets you trace a single order through all services in your log aggregator.

The LLM-specific metrics you must track per job: tokens_input, tokens_output, model_used, latency_ms, cost_usd, extraction_confidence_score, number_of_corrections_by_human. Aggregate these weekly per builder and per document type — this is how you identify which builder format has poor extraction quality and needs prompt improvement.